escola
britânica de
artes criativas
& tecnologia

QUALIDADE DE SOFTWARE

Anissa Ahmed

Análise de Qualidade

Florianópolis

2026

RESUMO

Este projeto teve como objetivo aplicar, de forma prática, os principais conceitos de Qualidade de Software por meio da automação de testes em diferentes camadas. Foram desenvolvidos testes automatizados de interface utilizando Cypress, contemplando a história de usuário de adição de produtos ao carrinho em uma aplicação web. Na camada de serviços, foram implementados testes de API para a funcionalidade de cupons, cobrindo os cenários de listagem e cadastro, com autenticação e validação das regras de negócio.

Também foi realizada a criação de testes de performance com Apache JMeter, simulando múltiplos usuários no fluxo de login, utilizando massa de dados dinâmica em arquivo CSV. Além disso, o projeto incluiu a configuração de integração contínua com Jenkins, por meio da criação de um Jenkinsfile versionado no GitHub, permitindo a execução automatizada dos testes.

O desenvolvimento deste trabalho possibilitou consolidar conhecimentos em automação UI, API, performance e CI/CD, além de reforçar a importância da qualidade para garantir confiabilidade, eficiência e escalabilidade das aplicações.

SUMÁRIO

RESUMO ***2***

INTRODUÇÃO ***4***

O PROJETO ***5***

1.1 Estratégia de teste **5**

1.2 Critérios de aceitação **6**

1.2.1 História de usuário 1: [US-0001] – Adicionar item ao carrinho **6**

1.2.2 História de usuário 2: [US-0002] – Login na plataforma **7**

1.2.3 História de usuário 2: [US-0003] – API de cupons **7**

1.3 Casos de testes **8**

1.3.1 História de usuário 1: **8**

CT01 – Adicionar até 10 itens do mesmo produto 8

1.3.2 História de usuário 2: **9**

CT01 – Login com email válido 9

CT02 – Login com CPF válido 9

CT03 – Bloqueio após 3 erros 9

1.1.1 História de usuário 3: API de Cupom **9**

CT01 – GET listar todos os cupons 9

CT02 – POST cadastrar cupom válido 10

CT03 – POST com nome de cupom duplicado 10

1.2 Repositório no Github **10**

1.3 Testes automatizados **10**

1.4 Integração contínua **11**

1.5 Testes de performance **11**

2. CONCLUSÃO ***13***

3. REFERÊNCIAS BIBLIOGRÁFICAS ***14***

INTRODUÇÃO

Este projeto tem como objetivo aplicar, de forma prática, os conhecimentos adquiridos ao longo do curso Engenheiro de Qualidade, contemplando todas as etapas do ciclo de testes de software.

A proposta consiste em simular a atuação de um profissional de QA inserido em um time ágil, sendo responsável pelo planejamento, definição de estratégias, criação de cenários e execução de testes em diferentes camadas da aplicação. O projeto abrange funcionalidades essenciais como autenticação de usuários, adição de itens ao carrinho e validação de cupons via API.

Ao longo do desenvolvimento, serão utilizadas boas práticas de qualidade de software, incluindo a definição de critérios de aceitação em linguagem Gherkin, elaboração de casos de teste baseados em técnicas como partição de equivalência e análise de valor limite, além da implementação de testes automatizados para interface web, APIs e testes de performance.

Também serão aplicadas ferramentas amplamente utilizadas no mercado, como Cypress para automação, Apache JMeter para testes de performance, Appium para testes mobile e Jenkins para integração contínua, garantindo uma abordagem moderna e alinhada às demandas reais da área de qualidade.

Espera-se, com este projeto, demonstrar não apenas a capacidade técnica na execução de testes, mas também a organização, clareza na documentação e visão estratégica necessárias para atuar como um profissional de QA, contribuindo para a entrega de um software confiável e de qualidade.

O PROJETO

Para o Projeto mestre da primeira parte do curso Profissão: Engenheiro de Qualidade, você deve considerar as histórias de usuário já refinadas e como se você estivesse participando de um time ágil. As funcionalidades devem seguir todo o fluxo de trabalho de um QA, desde o planejamento até a entrega. Siga as etapas dos sub-tópicos para te orientar no trabalho. Todas as boas práticas, tanto de documentação, escrita e desenvolvimento, serão consideradas na nota. Portanto caprichem, pois além de trabalho servir como nota para o curso, vai servir como Portfólio em seu github.

1.1 Estratégia de teste

- Faça uma estratégia de testes em um mapa mental, seguindo algumas diretrizes como objetivos, papeis e responsabilidades, fases de testes, padrões, tipos de testes, técnicas de testes, ambientes, ferramentas, abordagem (manual ou automatizado), framework ou ferramenta usados, plataformas (web, api, mobile), etc.;
- Referência: Módulo 5

1.2 Critérios de aceitação

- Considere as histórias de usuário: [US-0001] – Adicionar item ao carrinho, [US-0002] – Login na plataforma e [US-0003] – API de cupons
- Para cada uma delas crie pelo menos 2 critérios de aceitação usando a linguagem Gherkin;
- Em pelo menos um dos critérios, usar tabela de exemplos (Esquema do Cenário / Scenario Outline);
- Referência: Módulo 8

1.2.1 História de usuário 1: [US-0001] – Adicionar item ao carrinho

<u>Cenário 1: Adicionar produto ao carrinho com sucesso</u>

Dado que o cliente está na página do produto

Quando selecionar a quantidade 2

E clicar em "Adicionar ao carrinho"

Então o produto deve ser adicionado com sucesso

E o valor total deve ser atualizado

<u>Cenário 2: Aplicar desconto automático por faixa</u>

Dado que o cliente adicionou produtos ao carrinho

Quando o valor total for <valor>

Então o sistema deve aplicar o desconto de <desconto>

<u>Exemplos:</u>

| valor | desconto |

| 250 | 10% |

| 650 | 15% |

1.2.2 História de usuário 2: [US-0002] – Login na plataforma

Cenário 1: Login com usuário ativo

Dado que o usuário está na tela de login

Quando informar email e senha válidos

Então o sistema deve autenticar com sucesso

E redirecionar para a tela de pedidos

Cenário 2: Bloquear usuário após tentativas inválidas

Dado que o usuário está na tela de login

Quando informar senha inválida 3 vezes

Então o sistema deve bloquear o login por 15 minutos

1.2.3 História de usuário 2: [US-0003] – API de cupons

Cenário 1: Listar cupons cadastrados

Dado que o admin possui autenticação válida

Quando realizar uma requisição GET de cupons

Então o sistema deve retornar status 200

E listar todos os cupons cadastrados

Cenário 2: Cadastrar cupom com dados obrigatórios

Dado que o admin possui autenticação válida

Quando enviar um POST com código <código> e valor <valor>

Então o sistema deve cadastrar o cupom com sucesso

Exemplos:

| código | valor |

| Ganhe10 | 10 |

| Promo15 | 15 |

1.3 Casos de testes

- Crie pelo menos 3 casos de testes para cada história de usuário, sempre que possível, usando as técnicas de testes (partição de equivalência, valor limite, tabela de decisão etc.).
- Considere sempre o caminho feliz (fluxo principal) e o caminho alternativo e negativo (fluxo alternativo). Exemplo de cenário negativo: "Ao preencher com usuário e senha inválidos deve exibir uma mensagem de alerta..."
- Referência: Módulo 4 e 5

1.3.1 História de usuário 1:

CT01 – Adicionar até 10 itens do mesmo produto

Técnica: Valor limite
Pré-condição: Cliente na página do produto
Passos:

1. Selecionar quantidade 10
2. Clicar em adicionar ao carrinho
   Resultado esperado: Produto adicionado com sucesso.

CT02 – Adicionar 11 itens do mesmo produto

Técnica: Valor limite (negativo)
Pré-condição: Cliente na página do produto
Passos:

1. Selecionar quantidade 11
2. Clicar em adicionar
   Resultado esperado: Exibir mensagem de limite máximo de 10 itens.

CT03 – Validar cupom automático de 15%

Técnica: Partição de equivalência
Pré-condição: Carrinho com valor acima de R$ 600
Passos:

1. Adicionar produtos até totalizar R$ 650
   Resultado esperado: Aplicar cupom automático de 15%.

1.3.2 História de usuário 2:

CT01 – Login com email válido

Técnica: Fluxo principal
Pré-condição: Usuário ativo cadastrado
Passos:

1. Informar email válido
2. Informar senha válida
3. Clicar em Entrar
   Resultado esperado: Login realizado com sucesso.

CT02 – Login com CPF válido

Técnica: Partição de equivalência
Pré-condição: Usuário ativo cadastrado
Passos:

1. Informar CPF válido
2. Informar senha válida
   Resultado esperado: Sistema autentica normalmente.

CT03 – Bloqueio após 3 erros

Técnica: Tabela de decisão
Pré-condição: Usuário ativo
Passos:

1. Informar senha inválida 3 vezes
   Resultado esperado: Bloqueio por 15 minutos.

1.1.1 História de usuário 3: API de Cupom

CT01 – GET listar todos os cupons

Técnica: Fluxo principal
Pré-condição: API disponível
Passos:

1. Enviar GET /coupons
   Resultado esperado: Status 200 e lista retornada.

CT02 – POST cadastrar cupom válido

Técnica: Partição de equivalência
Pré-condição: Admin autenticado
Passos:

1. Enviar POST com campos obrigatórios válidos
   Resultado esperado: Status 201 e cupom criado.

CT03 – POST com nome de cupom duplicado

Técnica: Negativo / regra de negócio
Pré-condição: Cupom já existente
Passos:

1. Tentar cadastrar o mesmo código novamente
   Resultado esperado: Exibir erro de cupom duplicado.

1.2 Repositório no Github

- Crie um repositório no github com o nome TCC-EBAC;
- Deixe o repositório publico até a análise dos tutores;
- Neste repositório você deve subir este arquivo e todos os código fontes da automação WEB, API, Mobile, Performance e CI.
- Referência: Módulo 10
- Link do repositório: https://github.com/Anissa-Qa/TCC-EBAC
-

1.3 Testes automatizados

1.3.1 Automação de UI

- Crie um projeto de automação no Cypress;
- Crie uma pasta chamada UI para os testes WEB da História de Usuário [US-0001] – Adicionar item ao carrinho;
- Na automação deve adicionar pelo menos 3 produtos diferentes e validar se os itens foram adicionados com sucesso.

1.3.2 Automação de API

- Crie uma pasta chamada API para os testes de API da História de usuário **“Api de cupons”**.
- Faça a automação de **listar** os cupons e **cadastrar** cupom, seguindo as regras da História de usuário.
- Exemplo da automação de Api – GET

```
it('Deve listar todos os cupons cadastrados', () => {
  cy.request({
```

```
        method: 'GET',
        url: 'coupons',
        headers: {
            authorization: 'código_da_autorização_aqui'
        }
    }).should((response) => {
        cy.log(response)
        expect(response.status).to.equal(200)
    })
});
```

- Obs.: Considere todas as boas práticas de otimização de cenários (Page Objects, Massa de dados, Custom Commands, elementos etc.).
- Referência: Módulo 11, 12 e 14

1.4 Integração contínua

- Coloque os testes automatizados na integração contínua com jenkins, criando um job para execução da sua automação;
- Compartilhe o *jenkinsfile* no repositório, junto ao seu projeto.
- Referência: Módulo 15

1.5 Testes de performance

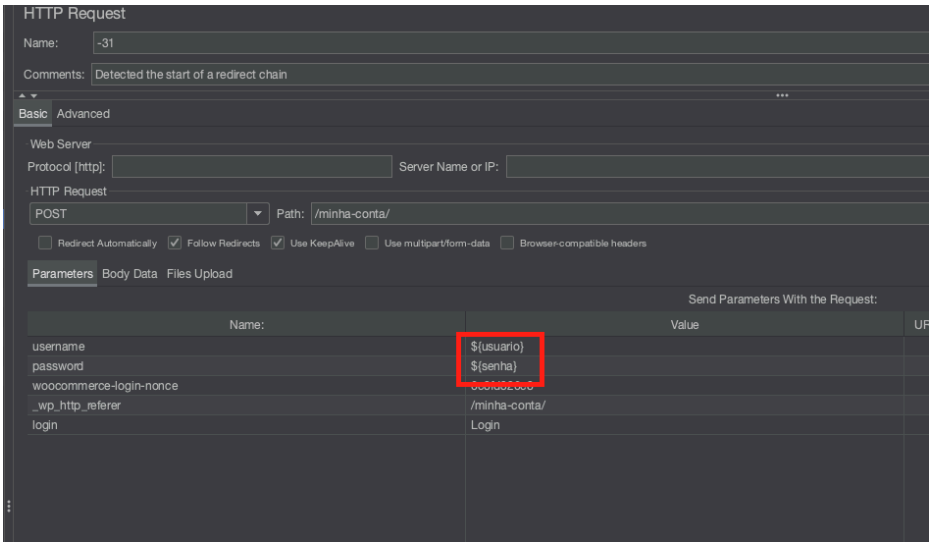
- Usando o Apache Jmeter, faça um teste de performance com o fluxo de login da História de usuário: [US-0002] – Login na plataforma
- Crie um template de gravação no jmeter (recording);
- Use massa de dados dinâmica em arquivo CSV;
- Referência: Módulo 18
- Configurações do teste de performance:

  -Usuários virtuais: 20
  -Tempo de execução: 2 minutos
  -RampUp: 20 segundos
  -Massa de dados: Usuário / senha:

  user1_ebac / psw!ebac@test
  user2_ebac / psw!ebac@test
  user3_ebac / psw!ebac@test
  user4_ebac / psw!ebac@test
  user5_ebac / psw!ebac@test

Nome de usuário	Nome	E-mail	Função
user1_ebac	–	user1_ebac@ebac.com	Assinante
user2_ebac	–	user2_ebac@ebac.com	Assinante
user3_ebac	–	user3_ebac@ebac.com	Assinante
user4_ebac	–	user4_ebac@ebac.com	Assinante
user5_ebac	–	user5_ebac@ebac.com	Assinante

- DICA: Em uma das requisições, após a gravação, vai aparecer os parâmetros usado. Substitua esses parâmetros pela sua massa de dados, conforme aprendido em aula:



2. CONCLUSÃO

A realização deste trabalho foi uma experiência extremamente enriquecedora para minha formação profissional na área de Qualidade de Software. Ao longo do desenvolvimento do projeto, pude aplicar na prática conceitos fundamentais estudados durante o curso, como automação de testes de interface, testes de API, testes de performance e integração contínua.

Um dos maiores aprendizados foi compreender a importância de estruturar cenários de teste de forma organizada, reutilizável e escalável, utilizando boas práticas como custom commands, separação por camadas e massa de dados dinâmica. Também foi muito valioso perceber como pequenos detalhes de configuração, seletores e ambiente podem impactar diretamente no sucesso da automação, exigindo análise crítica e capacidade de resolver pequenos bugs que aparecem.

Na parte de testes de API, aprendi a validar serviços de forma mais estratégica, garantindo não apenas o status code, mas também regras de negócio, autenticação e integridade dos dados. Já com o JMeter, pude desenvolver uma visão mais ampla sobre o comportamento da aplicação sob carga e a relevância dos testes de performance na prevenção de falhas em produção.

A implementação do Jenkins reforçou a importância da integração contínua dentro do processo de qualidade, demonstrando como a automação pode contribuir para entregas mais rápidas, seguras e confiáveis.

Como lição para minha vida profissional, levo a importância da persistência na resolução de problemas, da análise detalhada dos erros e da construção de soluções escaláveis. Este projeto fortaleceu minha confiança para atuar com testes automatizados em diferentes camadas e mostrou, na prática, como a qualidade é essencial para a evolução de produtos digitais.

3. REFERÊNCIAS BIBLIOGRÁFICAS

EBAC. Curso Profissão: Engenheiro de Qualidade de Software. Material didático dos módulos 11, 12, 14 e 18. Escola Britânica de Artes Criativas e Tecnologia, 2026.

CYPRESS. Cypress Documentation. Disponível em: https://docs.cypress.io/.

APACHE. Apache JMeter User Manual. Disponível em: https://jmeter.apache.org/usermanual/index.html.

JENKINS. Jenkins User Documentation. Disponível em: https://www.jenkins.io/doc/.

WOOCOMMERCE. REST API Documentation. Disponível em: http://lojaebac.ebaconline.art.br/rest-api/docs/.

GITHUB. GitHub Documentation. Disponível em: https://docs.github.com/.